Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System

CO4-ASSESSMENT 2

Course:

Software Engineering / Code Review and Version Control Evaluation

Name:

Davis Damien M

192521124

1. Introduction

The Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System is a software platform designed to automate and improve hotel operations and guest experiences using Artificial Intelligence.

The system can provide features such as:

- AI-based guest assistance/chatbot
- Online room booking and reservation management
- Check-in/check-out management
- Room availability and housekeeping management
- Guest feedback and sentiment analysis
- Personalized recommendations
- Hotel staff dashboard
- Notifications and alerts
- Payment integration
- AI-based demand forecasting and analytics

Since the system handles guest information, reservations, payments, and AI services, continuous integration and deployment are important to ensure that new features can be released quickly while maintaining security, reliability, and software quality.

The proposed CI/CD pipeline automates the complete process from developer code commit to production deployment.

---

2. CI/CD Requirements Analysis

2.1 Project Requirements

The proposed system requires:

Requirement	CI/CD Need
Frequent feature updates	Automated CI pipeline
AI/ML model updates	Automated model testing and validation
Web/mobile/API components	Automated build and testing
Guest data protection	Security scanning
Reliable hotel operations	Automated regression testing
High availability	Safe deployment and rollback
Multiple developers	Source-code management
Production reliability	Staging environment
Fast bug detection	Automated testing
Maintainable code	Code quality analysis
Continuous monitoring	Production monitoring and alerts

2.2 Why CI/CD is Important

Without CI/CD, developers would need to manually build, test, and deploy the application. This can result in:

- Human errors
- Inconsistent deployments
- Delayed releases
- Undetected bugs
- Security vulnerabilities
- Difficult rollback procedures

CI/CD provides an automated and repeatable deployment process.

---

3. Proposed Technology Stack

A suitable technology stack for the project is:

Component	Proposed Technology
Source Code Management	Git + GitHub
Backend	Python / FastAPI
Frontend	React
AI/ML	Python, scikit-learn / PyTorch
Database	PostgreSQL
Containerization	Docker
CI/CD	GitHub Actions
Code Quality	SonarQube
Unit Testing	PyTest, Jest
API Testing	Postman/Newman
Security Scanning	Trivy, Dependabot
Package Registry	GitHub Container Registry
Deployment	Kubernetes
Infrastructure	Cloud platform
Monitoring	Prometheus + Grafana
Logging	ELK Stack
Notifications	Email / Slack / Microsoft Teams

Justification

GitHub provides source-code management, pull requests, branch protection, and integration with GitHub Actions.

GitHub Actions is suitable because it can automate building, testing, security analysis, Docker image creation, and deployment within one workflow.

Docker ensures that the application behaves consistently across development, testing, and production.

Kubernetes provides scalability and supports rolling deployments, health checks, and rollback.

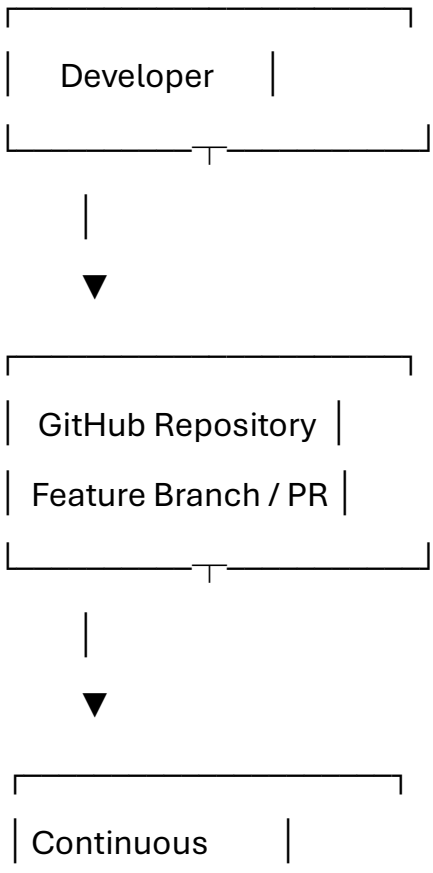
SonarQube identifies bugs, vulnerabilities, code smells, and maintainability problems.

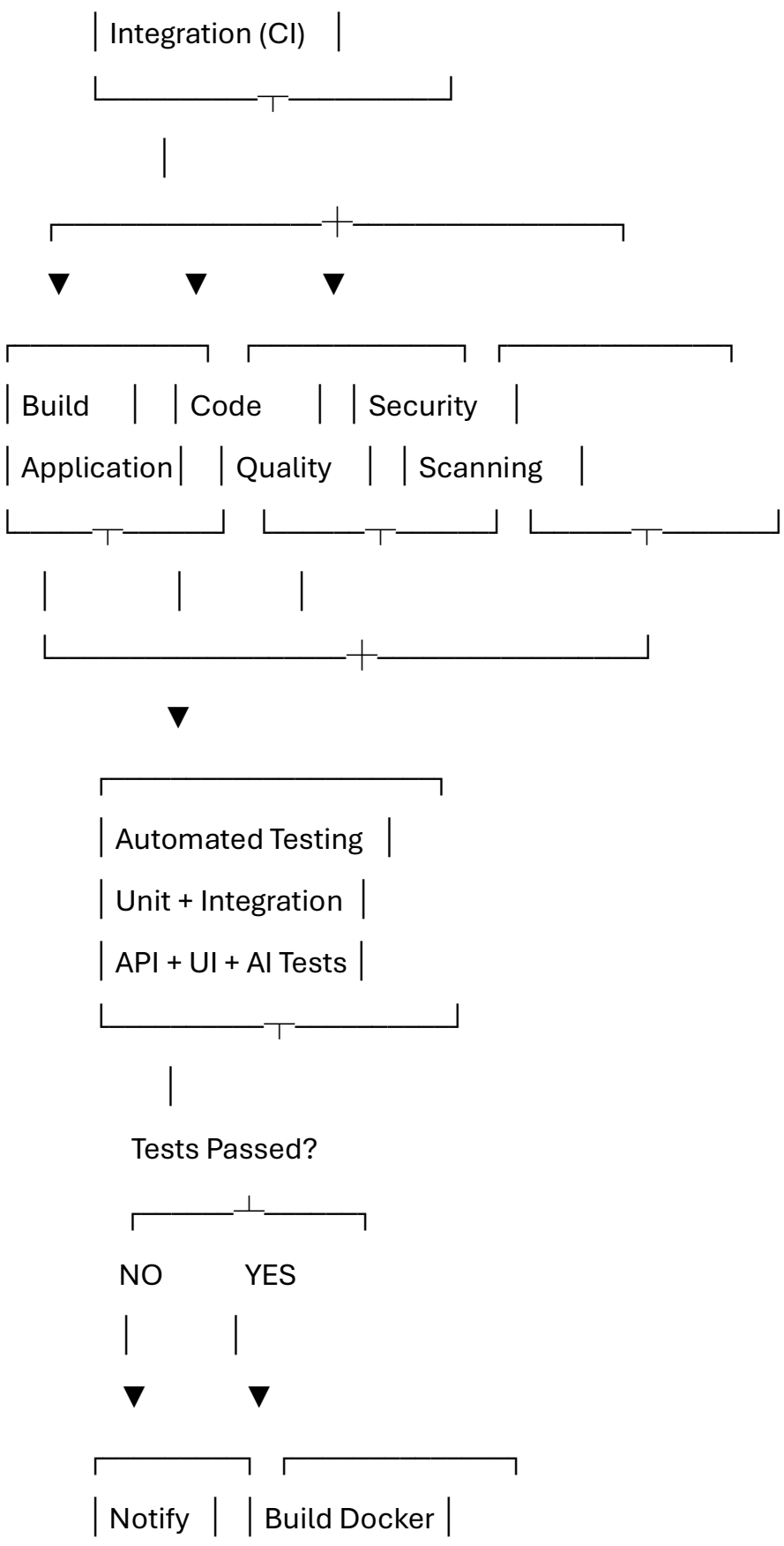
PyTest/Jest provide automated testing for backend and frontend components.

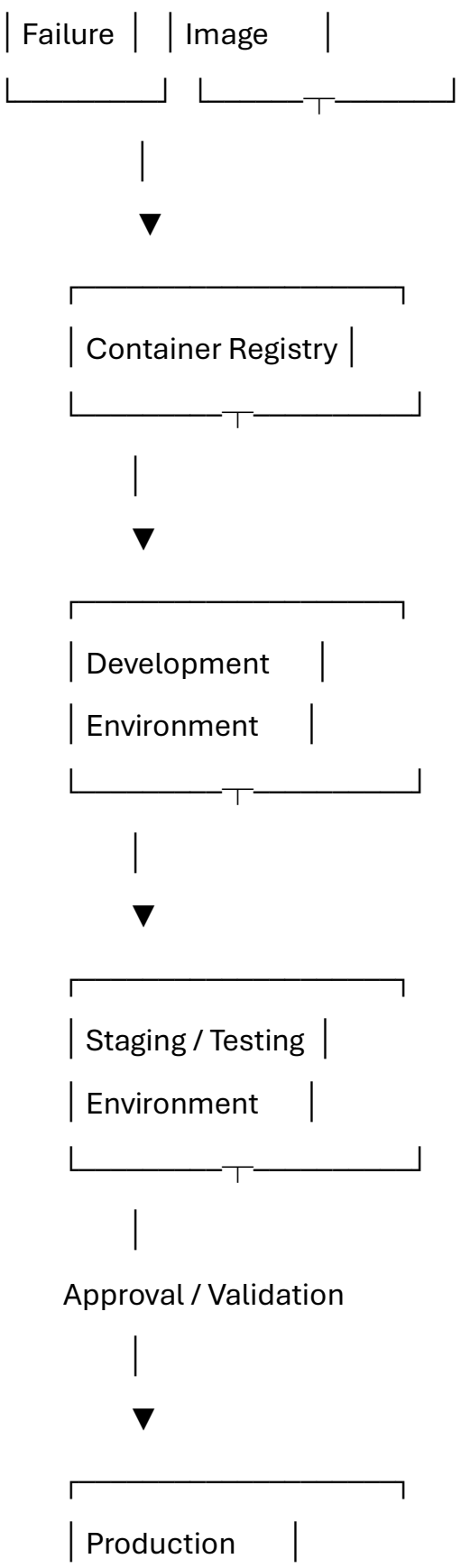
---

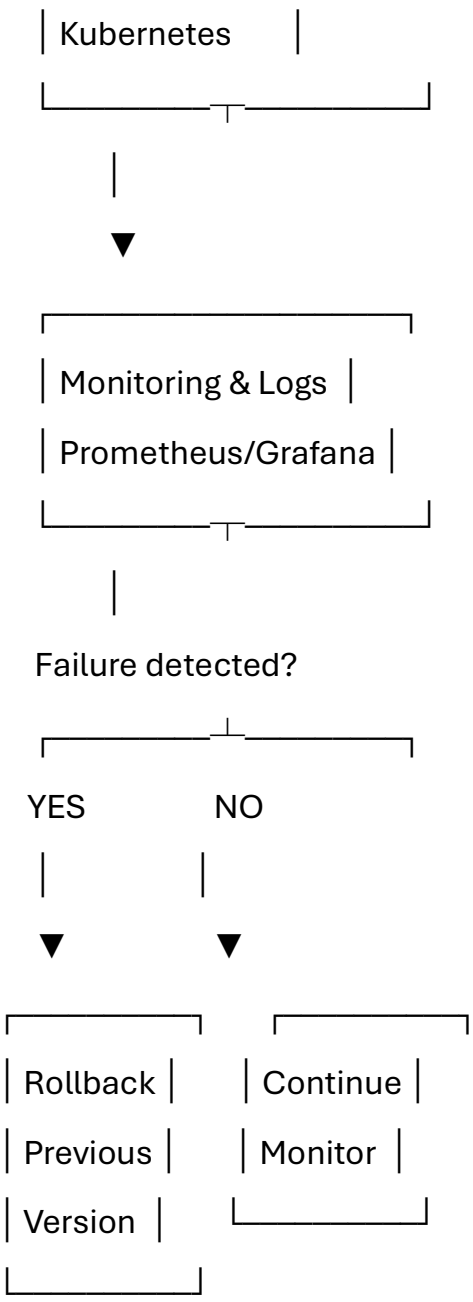
4. CI/CD Pipeline Architecture

The proposed pipeline is:




Integration (CI)
Build Application
Code Quality
Security Scanning
Automated Testing
Unit + Integration
API + UI + AI Tests
Tests Passed?
NO
YES
Notify
Build Docker


Failure
Image
Container Registry
Development Environment
Staging / Testing Environment
Approval / Validation
Production



---

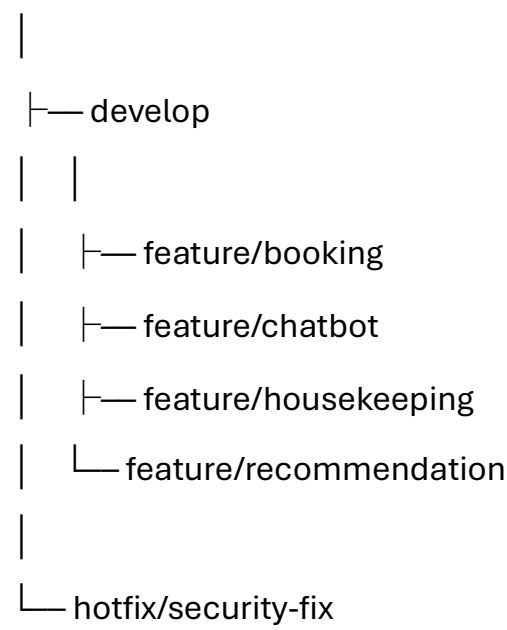
5. Pipeline Stages

Stage 1: Source Code Management

Developers work using Git.

A recommended branching strategy is:

main



Workflow

1. Developer creates a feature branch.
2. Developer implements the feature.
3. Developer commits the changes.
4. Pull request is created.
5. CI pipeline automatically starts.
6. Code review is performed.
7. Only successfully tested code is merged.

Quality Control

The main branch should have:

- Required pull-request reviews
- Status checks
- Branch protection
- No direct commits
- Successful CI pipeline before merging

---

6. Build Stage

After a commit or pull request, the pipeline builds the application.

Backend

Install Python

↓

Create virtual environment

↓

Install dependencies

↓

Compile/check application

Frontend

Install Node.js

↓

npm install

↓

npm run build

AI Module

The pipeline verifies:

- Python dependencies
- ML libraries
- Model-loading code
- Model configuration
- Data-processing components

A failed build stops the pipeline.

---

7. Automated Testing Strategy

Testing is one of the most important stages of the pipeline.

7.1 Unit Testing

Unit tests validate individual functions and modules.

Examples:

Reservation calculation

Room availability

Guest registration

Login validation

Recommendation function

Payment calculation

Tools:

- PyTest
- Jest

Example:

Test: Book available room

Input: Room 205, 2 nights

Expected: Booking successfully created

7.2 Integration Testing

Integration tests verify communication between different components.

For example:

Frontend

↓

REST API

↓

Backend

↓

PostgreSQL

Test cases include:

- API ↔ database
- Booking service ↔ room management
- Chatbot ↔ AI service
- Payment service ↔ booking service

7.3 API Testing

APIs should be automatically tested after the application is built.

Example:

POST /api/login

POST /api/bookings

GET /api/rooms

PUT /api/rooms/{id}

POST /api/feedback

Expected HTTP responses and data formats are validated.

Postman/Newman can be used for automated API testing.

7.4 UI Testing

Automated UI tests verify important guest and staff workflows.

Examples:

Guest Login

↓

Search Room

↓

Select Room

↓

Make Reservation

↓

Receive Confirmation

Other tests:

- Guest check-in
- Guest check-out
- Cancellation
- Staff login
- Housekeeping status update

Tools such as Playwright can automate browser testing.

---

8. AI/ML Testing

Because this is an AI-based hotel system, traditional software testing alone is insufficient.

The pipeline should test AI components for:

Model Accuracy

The pipeline evaluates whether model accuracy remains above a predefined threshold.

For example:

Previous model accuracy = 91%

New model accuracy = 92%

Minimum acceptable = 88%

Result = PASS

Model Regression Testing

The new model is compared against the previous production model.

New Model

↓

Test Dataset

↓

Performance Metrics

↓

Compare with Production Model

↓

Accept / Reject

AI Functional Testing

Test whether:

- Chatbot returns valid responses
- Recommendations are relevant
- Sentiment analysis produces valid classifications
- Forecasting produces reasonable results

A major degradation should stop deployment.

---

9. Code Quality Analysis

After building the application, the pipeline performs static code analysis.

Tool: SonarQube

SonarQube checks:

- Bugs
- Vulnerabilities
- Code smells
- Duplicated code
- Maintainability
- Test coverage

Pipeline:

Source Code

↓

SonarQube Scan

↓

Quality Gate

↓

PASS ——————► Continue

FAIL ——————► Stop Pipeline

A quality gate prevents poor-quality code from reaching production.

---

10. Security Checks

Security is particularly important because the system handles guest and booking information.

The pipeline should include:

Dependency Scanning

Identify vulnerable third-party packages.

Example:

Python dependency

↓

Vulnerability scan

↓

Vulnerability found?

↓

YES → Pipeline fails

Container Security

Docker images should be scanned using Trivy.

It checks for:

- OS vulnerabilities
- Vulnerable packages
- Known CVEs
- Configuration issues

Secret Detection

The pipeline should detect accidentally committed:

- Passwords
- API keys
- Database credentials
- Authentication tokens

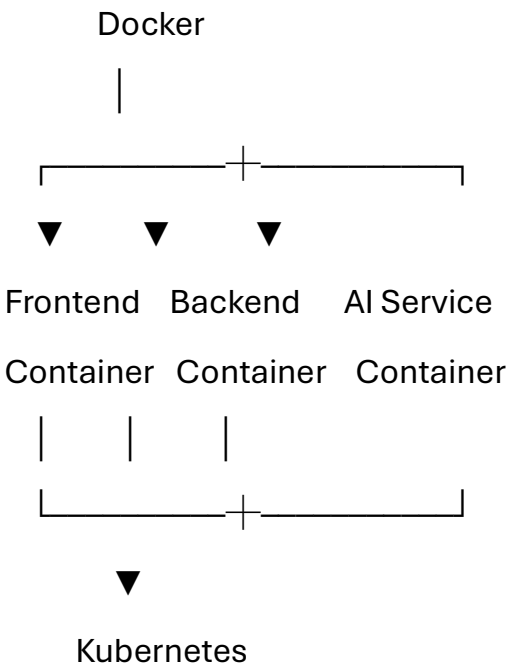
Secrets should instead be stored using GitHub Secrets or a cloud secret manager.

---

11. Packaging

After all tests pass, the application is packaged into Docker containers.

Example architecture:



Docker images are tagged using a version number.

Example:

hotel-frontend:v1.4.0

hotel-backend:v1.4.0

hotel-ai:v1.4.0

The images are pushed to GitHub Container Registry.

---

12. Deployment Environments

Three primary environments are recommended.

12.1 Development

Purpose:

- Developer testing
- Feature validation
- Early integration testing

Feature Branch

↓

Development

12.2 Staging

Staging should closely resemble production.

Purpose:

- Integration testing
- End-to-end testing
- Performance testing
- Security validation
- User acceptance testing

Development

↓

Staging

↓

Validation

12.3 Production

Production is the live hotel system.

It handles:

- Real guest interactions
- Reservations
- Hotel staff operations
- AI services

- Notifications

Production deployment should occur only after all required quality gates pass.

13. Deployment Strategy

A rolling deployment can be used for the production environment.

Example:

Old Version

Pod 1 ———— v1.3

Pod 2 ———— v1.3

Pod 3 ———— v1.3

↓ Deployment

Pod 1 ———— v1.4

Pod 2 ———— v1.3

Pod 3 ———— v1.3

↓

Pod 1 ———— v1.4

Pod 2 ———— v1.4

Pod 3 ———— v1.3

↓

Pod 1 ———— v1.4

Pod 2 ———— v1.4

Pod 3 ———— v1.4

This reduces downtime because old instances remain available while new instances are deployed.

For particularly critical releases, blue-green or canary deployment can also be used.

14. Rollback Mechanism

If the new release causes problems, Kubernetes can return to the previous stable version.

Example:

Production

↓

v1.4 deployed

↓

Health check fails

↓

Monitoring detects problem

↓

Rollback triggered

↓

v1.3 restored

↓

Service continues

Rollback conditions may include:

- Application crash
- Failed health checks
- High error rate
- Significant performance degradation
- Failed database migration
- AI model performance below threshold

15. Monitoring and Notifications

After deployment, the system must continuously monitor application health.

Prometheus

Collects metrics such as:

- CPU usage
- Memory usage
- Request rate
- Error rate
- Response time

Grafana

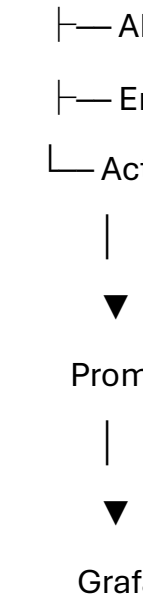
Provides dashboards for hotel system administrators and DevOps teams.

Example:

Application

├— CPU

├— Memory



Notifications

The team can receive notifications when:

- Build succeeds
- Build fails
- Deployment succeeds
- Deployment fails
- Security vulnerability is detected
- Production health check fails

Notifications can be sent through email or a team collaboration platform.

16. Example GitHub Actions Workflow

A simplified CI/CD workflow can be represented as:

```
name: Hotel-Management-CI-CD


on:
  push:
```

```
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:

  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run unit tests
        run: pytest

      - name: Code quality analysis
```

```
      run: sonar-scanner

    - name: Security scan
      run: trivy fs .

    - name: Build Docker image
      run: docker build -t hotel-backend:${{ github.sha }} .

    - name: Push Docker image
      run: docker push hotel-backend:${{ github.sha }}

deploy-staging:
  needs: build
  runs-on: ubuntu-latest

  steps:
    - name: Deploy to staging
      run: kubectl apply -f kubernetes/staging/

deploy-production:
  needs: deploy-staging
  runs-on: ubuntu-latest

  environment:
    name: production
```

```
  steps:
    - name: Deploy to production
      run: kubectl apply -f kubernetes/production/
```

This is a simplified academic example; a real pipeline would also configure credentials, image registries, environment-specific variables, test reports, health checks, and deployment verification.

17. Complete Pipeline Workflow

The complete workflow is:

1. Developer writes code

↓

2. Commit to GitHub

↓

3. Pull Request created

↓

4. Build application

↓

5. Dependency installation

↓

6. Unit tests

↓

7. Integration tests

↓

8. API/UI tests

↓

9. AI/ML model tests

↓

10. SonarQube quality analysis

↓

11. Security & dependency scanning

↓

12. Build Docker image

↓

13. Scan Docker image

↓

14. Push image to registry

↓

15. Deploy to Development

↓

16. Deploy to Staging

↓

17. End-to-end testing

↓

18. Approval / Quality Gate

↓

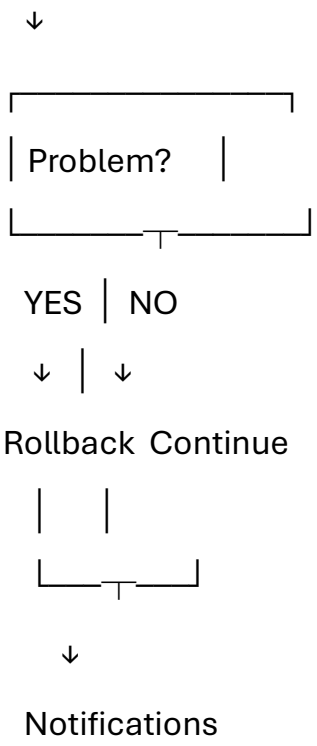
19. Deploy to Production

↓

20. Health checks

↓

21. Monitoring



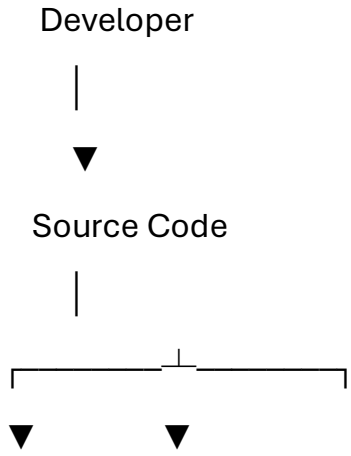
18. Tool Selection and Justification

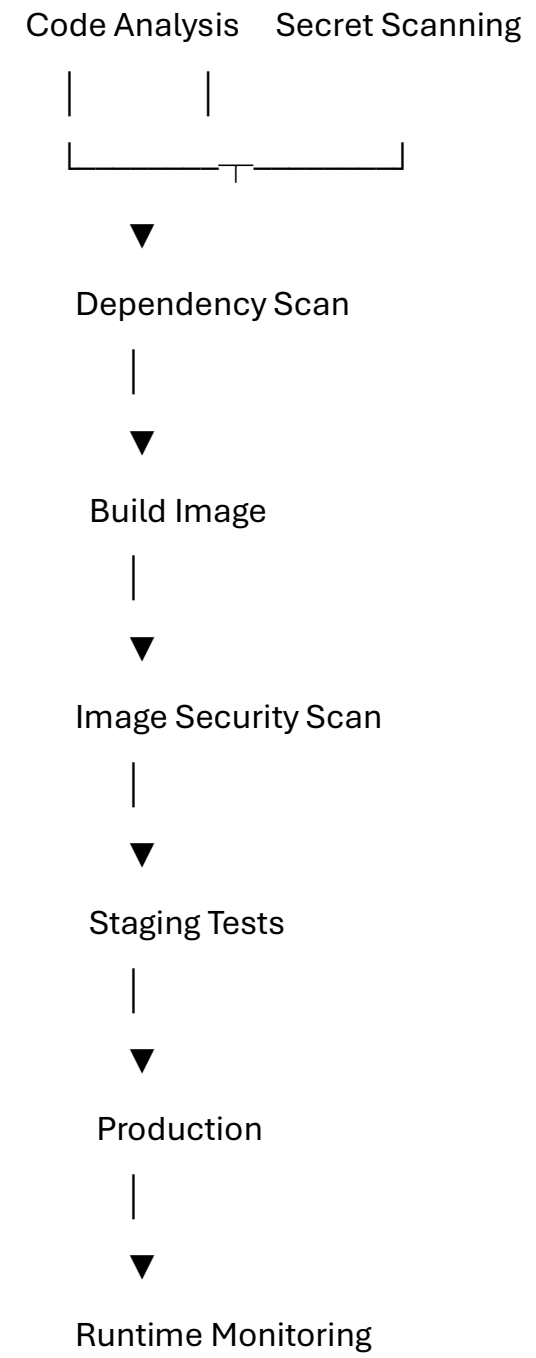
Pipeline Activity	Tool	Justification
Source control	GitHub	Version control and collaboration
CI/CD	GitHub Actions	Easy automation and GitHub integration
Backend testing	PyTest	Popular Python testing framework
Frontend testing	Jest	JavaScript/React testing
UI testing	Playwright	Automated browser testing
API testing	Postman/Newman	Convenient API automation
Code quality	SonarQube	Detects bugs and code smells

Pipeline Activity	Tool	Justification
Dependency security	Dependabot	Identifies vulnerable dependencies
Container security	Trivy	Lightweight Docker vulnerability scanner
Packaging	Docker	Consistent application environments
Registry	GitHub Container Registry	Stores versioned images
Deployment	Kubernetes	Scalable container orchestration
Monitoring	Prometheus	Application/infrastructure metrics
Visualization	Grafana	Monitoring dashboards
Logging	ELK	Centralized logs
Notifications	Email/Teams/Slack	Immediate failure alerts

19. Security Architecture

The CI/CD pipeline should follow a DevSecOps approach.





Security controls include:

- HTTPS/TLS
- Authentication and authorization
- Role-based access control
- Secret management

- Dependency scanning
- Container scanning
- Input validation
- Database access controls
- Audit logging
- Regular vulnerability scanning

20. Database Deployment Considerations

The hotel system will use a database containing information such as:

- Room information
- Reservations
- Guest profiles
- Staff information
- Feedback
- AI-related data

Database changes should therefore be managed using database migration tools.

Example:

Developer changes database schema

↓

Create migration

↓

Run migration tests

↓

Staging migration

↓

Production migration

Database backups should be performed regularly.

Production database changes should also be designed to support rollback or backward compatibility wherever possible.

21. Quality Gates

The pipeline should not deploy if important quality criteria fail.

Example quality gates:

Check	Minimum Requirement
Build	Must pass
Unit tests	100% must pass
Integration tests	Must pass
API tests	Must pass
Security scan	No critical vulnerabilities
Code quality	SonarQube quality gate passes
AI model	Meets predefined performance threshold
Docker scan	No critical vulnerabilities
Staging	End-to-end tests pass
Health check	Application healthy

22. Advantages of the Proposed CI/CD Pipeline

Faster Development

Developers receive rapid feedback when code contains errors.

Better Software Quality

Automated testing and quality analysis detect problems early.

Improved Security

Security scanning is integrated directly into development.

Reduced Deployment Errors

Automated deployment avoids many manual configuration mistakes.

Faster Releases

New features can be released frequently.

Easy Rollback

Kubernetes enables rapid recovery from unsuccessful deployments.

Scalability

Containerized services can be scaled based on hotel traffic.

Reliability

Automated health checks and monitoring improve system availability.

23. Conclusion

The proposed CI/CD pipeline provides an automated and secure software delivery process for the Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System.

The pipeline begins with GitHub-based source-code management, followed by automated building, unit testing, integration testing, API/UI testing, AI model validation, code-quality analysis, security scanning, and Docker packaging.

After successful validation, the application is deployed first to development and staging environments before being released to production. Kubernetes provides scalable deployment and rollback capabilities, while Prometheus and Grafana provide continuous monitoring.

Overall, the proposed CI/CD architecture enables the hotel system to achieve:

Faster development + automated testing + improved security + reliable deployment + continuous monitoring + rapid rollback

This makes the system more suitable for a real-world hotel environment where availability, security, data integrity, and guest experience are critical.